

Virtual Memory — Detailed Lecture Notes

JKSSB Computer Awareness

October 8, 2026

09.1 What virtual memory is and why it exists

Physical RAM is limited. A modern application, its data, and the other programs open at the same time can together demand more memory than the machine physically has. **Virtual memory** is the technique that lets the system keep running anyway: it uses secondary storage (the disk) to hold parts of a program that are not currently needed in RAM. In this way the *addressable* memory seen by a program can be much larger than the physical memory actually installed.

Definition (Virtual memory). *A memory-management technique that uses secondary storage to extend the addressable memory beyond physical RAM, giving each process the impression of a large, contiguous memory space.*

The office-desktop running example (EX-01) makes this concrete. The desktop has 16 GB of RAM and runs MS Word, a spreadsheet, and a web browser at once. If the combined working set of those programs exceeds 16 GB, the operating system moves some of the least-used pages to disk and brings them back when they are referenced again. No single program is told that RAM ran out; each continues to use its own large address space.

Two ideas are worth separating at the start:

- Virtual memory extends **capacity**. It lets a program use more memory than the physical RAM provides.
- Virtual memory does **not** add speed. Because it relies on the disk, it is **much slower** than RAM.

09.2 Virtual addresses, physical addresses, and the MMU

A program does not work with the real location of its data in RAM. It works with **virtual addresses**, which form a large, uniform address space. The hardware translates each virtual address into the actual **physical address** in RAM when the instruction executes.

Definition (Virtual address). *The address generated by a running program; it belongs to the program's large virtual address space and is not the real RAM location.*

Definition (Physical address). *The actual location of data in main memory (RAM), which the hardware uses after translation.*

The translator is the MMU, the Memory Management Unit.

Definition (MMU (Memory Management Unit)). *The hardware that translates virtual addresses into physical addresses at run time, using the page table maintained by the operating system.*

The MMU is central to every memory access. It does *not* store files and it does not itself create storage; it maps one address form to another. Keeping this role straight avoids two common exam errors: attributing file storage to the MMU, or describing translation as a software-only operation with no hardware component.

Example 09.2.1. *A question asks what the MMU does. The correct answer is that it translates virtual addresses into physical addresses. An option saying that the MMU stores files permanently is wrong: file storage is the job of secondary storage, not the address translator. An option saying the MMU holds the BIOS is also wrong; that describes firmware, not the unit that maps addresses.*

09.3 Paging: fixed-size pages and frames

Paging divides memory into equal, fixed-size blocks. The virtual address space of a process is split into **pages**, and physical memory is split into **frames** of exactly the same size. Any page can be placed in any free frame, and the page table records which page currently sits in which frame.

Definition (Page). *A fixed-size block of the virtual address space in paging.*

Definition (Frame). *A fixed-size block of physical memory (RAM) into which a page is loaded; frames are the same size as pages.*

Because every block is the same size, paging makes memory allocation simple and predictable. There is no awkward leftover space caused by blocks of many different sizes. The cost is a small amount of unused space in the last page of a program, but the mechanism itself is uniform.

Two related terms must not be swapped. **Pages** are the *virtual* blocks; **frames** are the *physical* blocks. They are the same size, but a page lives in the program's virtual view while a frame lives in RAM.

09.4 Segmentation: variable-size logical segments

Segmentation divides a program into **variable-size segments**, each corresponding to a logical unit of the program: the code, the data, the stack, a procedure, or a module. Because these logical units differ in size, segment sizes differ as well.

Definition (Segment). *A variable-size block of a program that corresponds to a logical unit such as code, data, stack, or a module.*

Segmentation reflects how a programmer thinks about a program: one segment per logical part. Paging, by contrast, reflects no logical unit at all — its pages are simply equal slices chosen for uniform allocation. This is the key near-neighbour distinction between the two techniques.

Example 09.4.1. *A stem asks which technique divides a program according to logical units such as code and data. The answer is segmentation, because those units are logical and of different sizes. If the stem instead describes equal fixed-size blocks and the terms “page” and “frame”, the answer is paging.*

09.5 Paging versus segmentation at a glance

Feature	Paging	Segmentation
Block size	Fixed	Variable
Blocks	Pages (virtual) and frames (physical)	Logical segments
Sizes equal?	Yes, all equal	No, they differ
Logical view	Not visible to the programmer	Matches program structure
Unit	Equal slices	Code, data, stack, module

The single most common error is to call paging's blocks "segments" or to say that segmentation uses equal-sized blocks. Keep the pairing firm: *paging is fixed-size; segmentation is variable-size and logical*.

09.6 Page faults and demand paging

When a program references a page that is *not* currently in RAM, the hardware raises a **page fault**. The operating system then loads the required page from the disk into a free frame and lets the program continue.

Definition (Page fault). *An event that occurs when a referenced page is not currently in RAM; the page must be loaded from secondary storage (disk) into a free frame.*

Definition (Demand paging). *A policy in which a page is loaded into memory only when it is actually referenced, rather than loading the whole program in advance.*

A page fault is a normal, expected event — **not a hardware fault**. It simply means the needed page is on the disk for the moment. Demand paging is what makes a program able to start before every one of its pages is in RAM: the remaining pages are brought in on demand, as execution reaches them.

Example 09.6.1. *A question says a running program references a page that is not in RAM and asks what happens. The correct answer is a page fault followed by loading that page from the disk. An option describing a physical hardware failure is wrong, because a page fault is a routine memory-management event, not a broken component.*

09.7 Swapping and the disk-backed hierarchy

Swapping is the movement of pages or whole processes between RAM and secondary storage (the disk). Because virtual memory depends on this movement, it is **disk-backed**: the disk supplies the extra capacity that RAM does not have.

Definition (Swapping). *Moving pages or processes between main memory (RAM) and secondary storage (disk) so that the memory actually needed at a moment can reside in RAM.*

The memory hierarchy, from fastest and smallest to slowest and largest, runs from CPU registers, to cache, to RAM, to disk-backed virtual memory. Two facts follow directly:

1. Virtual memory is **much slower than RAM**, because the disk is far slower than main memory.

2. Virtual memory is **not additional physical RAM**. It borrows disk space; it does not add RAM chips.

This is also where virtual memory is most often confused with **cache**. They are opposite in purpose and location. Cache is small, fast memory sitting close to the CPU and it *speeds up* repeated access to hot data. Virtual memory is large, disk-backed, and it *extends capacity* using slower storage. Cache helps speed; virtual memory helps size.

Example 09.7.1. *A stem contrasts cache and virtual memory. The correct statement is that cache is small, fast memory close to the CPU, while virtual memory is slower and disk-backed. Options that call both “secondary storage”, or that say virtual memory speeds up the CPU, reverse the roles and are wrong.*

09.8 Thrashing

Thrashing is *excessive paging*. When too many processes compete for too little RAM, the system spends most of its time moving pages between RAM and disk, and very little time doing useful work. The CPU stays busy, yet throughput collapses.

Definition (Thrashing). *A condition of excessive paging in which the system spends most of its time swapping pages between RAM and disk, leaving little time for useful work.*

Thrashing is a symptom of memory pressure, not a hardware failure. Adding physical RAM, or reducing the number of active processes, usually relieves it.

09.9 Exam retrieval and common confusions

Use two retrieval directions. Given the technique, recall its block rule; given a description, name the technique.

- Virtual memory ↔ extends memory using disk; slower than RAM; not extra physical RAM.
- MMU ↔ Memory Management Unit; translates virtual to physical addresses.
- Paging ↔ fixed-size pages (virtual) and frames (physical).
- Segmentation ↔ variable-size logical segments.
- Page fault ↔ referenced page not in RAM; load it from disk.
- Demand paging ↔ load a page only when it is referenced.
- Swapping ↔ move pages/processes between RAM and disk.
- Thrashing ↔ excessive paging.

Three distinctions prevent most errors in this topic:

1. Do not say virtual memory is **extra physical RAM**. It is disk-backed and **slower** than RAM; it adds capacity, not speed.
2. Do not confuse **paging** (fixed-size pages and frames) with **segmentation** (variable-size logical segments).
3. Do not call a **page fault** a hardware fault. It means the referenced page must be loaded from the disk; and do not confuse **cache** (small, fast, near the CPU) with virtual memory (large, disk-backed).

Exercises

1. Explain why virtual memory is said to extend memory but not to add physical RAM.
2. Expand MMU and state its function in one sentence.
3. Distinguish paging from segmentation in terms of block size and the unit being divided.
4. What is a page fault, and what happens after one occurs?
5. Define thrashing and give one practical way to reduce it.

Solutions

1. Virtual memory extends the addressable memory by using secondary storage (disk) to hold parts of a program not currently in RAM. It does not add RAM chips, so it is not additional physical RAM; because it is disk-backed, it is much slower than RAM.
2. MMU is the Memory Management Unit. It translates virtual addresses generated by a program into physical addresses in RAM at run time.
3. Paging uses fixed-size, equal blocks: pages in the virtual space and frames in physical memory. Segmentation uses variable-size blocks that correspond to logical program units such as code, data, or a module.
4. A page fault occurs when a program references a page that is not currently in RAM. The operating system then loads that page from the disk into a free frame and resumes execution.
5. Thrashing is excessive paging, in which the system spends most of its time swapping pages between RAM and disk and little time on useful work. Adding physical RAM (or reducing the number of active processes) usually reduces it.